centro UNISAL
Centro Universitário Salesiano de São Paulo
U.E. Campinas

Algoritmos e Fluxogramas

Lógica de Programação

Teoria

Introdução

Este curso tem o objetivo de desenvolver o que chamamos de lógica de programação em pessoas que queiram ingressar neste mundo ou profissionais que desejam desenvolver um pouco mais este quesito.

Quando falamos em lógica de programação, usamos vários termos como fluxograma, algoritmos, Pseudocódigo, entre outros.

No instante em que uma determinada tarefa passa a ser realizada por máquinas, inclusive computadores ao invés de ser realizada 100% pelo homem, estamos realizando um processo de automação.

Para que a automação de um determinado trabalho tenha êxito, é fundamental que o equipamento que irá realizar o mesmo tenha condições de desempenhar todas as etapas que o envolve com precisão e no menor espaço de tempo possível, garantindo também a repetição do processo, e tais instruções e solicitações deverão ser repassadas a esta máquina.

Esta especificação dos passos a serem seguidos e suas regras é dado o nome de algoritmo, ou seja, para que o computador possa realizar um determinado trabalho, será necessário que este seja detalhado passo a passo, através de uma forma compreensível pela máquina para ser empregado no que chamamos de programa.

Para representar um algoritmo, são utilizadas diversas técnicas e cabe ao profissional, adotar aquela que melhor se adapte as suas necessidades.

Existem algoritmos que apresentam os passos apenas a nível lógico, ou seja, não especifica detalhes de uma linguagem de programação em específico, e outros que tratam os passos a serem seguidos com maior riqueza de detalhes.

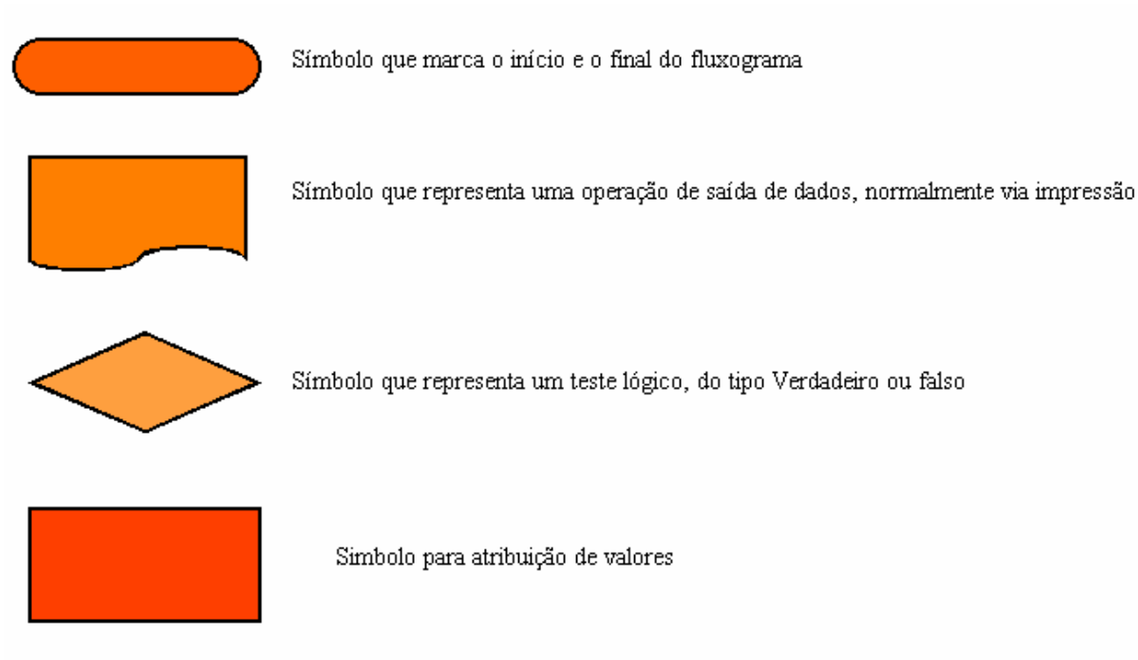
As formas mais conhecidas de algoritmos são:

- **Fluxograma** - representação gráfica de algoritmos, ou seja, das instruções e/ou módulos do processamento, conhecido também como diagrama de bloco;
- **Descrição narrativa** - expressa o algoritmo em linguagem natural, como por exemplo, as instruções de operação de um determinado eletrodoméstico, constantes do manual do operador;
- **Portugol (linguagem estruturada) -** forma de escrever algoritmos que muito se assemelha a forma na qual os programas são escritos nas linguagens de programação (Pascal, Cobol, Basic, dBASE, etc...).

Fluxogramas

Conforme citamos anteriormente, o fluxograma nada mais é do que uma representação gráfica do algoritmo, através de formas geométricas, facilitando a compreensão da lógica utilizada pelo profissional. Existem atualmente vários padrões para definir as formas geométricas a serem utilizadas para as diversas instruções (passos) a serem seguidos pelo sistema.

Abaixo, veja alguns símbolos que são utilizados no fluxograma:



Vamos a seguir, exibir um fluxograma que teria o objetivo de calcular o valor diário de um salário mínimo.

Seguindo uma lógica simples, teríamos que dividir o valor do salário mínimo (que sabemos ser equivalente a R$ 180,00) e dividir este total por 30 (número de dias de um mês).
Observe como ficaria este fluxograma:

Figura na Lousa

- A princípio marcamos o início do fluxograma;
- Em seguida, armazenamos em uma posição de memória (variável), o valor de 180,00. Esta posição de memória teve a denominação de “salariominimo”;
- Em seguida, armazenamos em uma posição de memória denominada “media” o valor da variável “salariominimo” dividido por 30;
- Em seguida, enviamos ao periférico de saída, o conteúdo da variável “media”
- Em seguida, finalizamos o fluxo.

Observe que da forma que o mesmo foi exposto, existe um procedimento lógico para o calculo proposto, bem como, poderemos repetir este procedimento por quantas vezes necessários, que o resultado sempre será correto e preciso.

Mais adiante, iremos criar fluxogramas mais complexos, expressando situações reais de processamento, este teve apenas o objetivo de lhe dar uma noção simples e objetiva de fluxogramas.

Portugol

Esta é uma forma de representar um algoritmo que ao contrário do fluxograma é mais rica em detalhes, se assemelhando muito as fontes (linhas de código) de uma linguagem de programação, onde temos que definir as variáveis, rotinas, sub-rotinas, etc... Ao invés de símbolos gráficos, utilizamos ordens (comandos) para solicitar uma determinada tarefa/rotina. Sua sintaxe básica é:

Algoritmo <*nome do algoritmo>*

<declaração das variáveis>

Inicio da rotina

<instruções a serem seguidas>

Fim da rotina

Vamos utilizar o mesmo exemplo e objetivo exposto na exemplificação do fluxograma, empregando o mesmo no Portugol.

```
Algoritmo Valor_dia
Var Salariominimo,media : real
Inicio
        SalarioMinimo=180,00
        Media=Salariominimo/30
        Envie para impressora “Media”
Fim
```

No exemplo acima, a cláusula VAR tem o objetivo de declarar as variáveis que serão usadas e o tipo REAL representa uma variável que pode assumir valores numéricos com parte decimal (mais adiante iremos estudar os principais tipos de variáveis).

Descrição Narrativa

Este tipo de representação é pouco utilizada para fins de desenvolvimento de programas de computador (algoritmos são usados em outros processos além deste), pois o uso da linguagem natural, pode proporcionar em diversos casos, uma interpretação errada do verdadeiro objetivo.
Por exemplo, a instrução “Acelere um pouco o carro”, no algoritmo de um jogo de corrida, poderá ser interpretado em 20Km por exemplo, sendo que o objetivo seria apenas 15Km, o que poderá gerar um erro de lógica no sistema.

Porém para manuais, este é o tipo de algoritmo ideal. Imagine um manual de instruções de uma televisão expresso por fluxograma, ao invés de:

Aperte o botão “Ligar” do seu controle remoto;
No quadro canais, pressione a tecla correspondente ao canal que quer assistir.

Quando trabalhamos com um programa de computador, podemos observar que durante todo o processo o equipamento realiza a manipulação de informações, através da entrada/leitura de uma série de dados. Este procedimento é realizado através de duas etapas distintas:

Instruções : comandos que determinam a forma pela qual os dados devem ser tratados.

Dados : são as informações recolhidas/fornecidas por diversos meios e que serão processados pelo computador através das instruções.

Em programas de computador, é realizada uma espécie de classificação de dados, através do tipo de informação que o mesmo representa.

Em termos gerais (sem abordar uma linguagem de programação em específico), temos os seguintes tipos de dados:

Dados numéricos

Os dados numéricos podem ser basicamente de dois tipos: inteiros e reais.

Os dados numéricos inteiros, conforme o próprio nome diz, não possuem partes decimais e podem representar valores negativos ou positivos. Como exemplo de números inteiros temos:

51 (número inteiro positivo);
0 (número inteiro positivo);
-51 (número inteiro negativo).

Já os dados numéricos reais são aqueles que podem possuir valores com partes decimais, podendo também ser negativos e/ou positivos. Como exemplo de números do tipo real temos:

24.52 (número real positivo com duas casas decimais);
-24.52 (número real negativo com duas casas decimais).

Vale lembrar que, ao exibir 0 (zero) estamos exibindo um dado numérico inteiro e ao exibir 0.0 (zero ponto zero) estamos exibindo um dado numérico real.

DADOS LITERAIS

Os dados do tipo literal são formados por um conjunto de caracteres que poderão ser formados por números, letras e símbolos especiais. Em linguagem de programação, na maioria das vezes, este dado é chamado de String. O conteúdo dos dados literais é demarcado pelo símbolo de aspas, em seu inicio e seu final. O tamanho de um dado literal é estipulado pelo comprimento dado pelo número de caracteres neste contido. Como exemplos de dados literais temos:

"Unisal - Ciência da Computação" (literal de comprimento 30)
"32.15" (literal de comprimento 5)

DADOS LÓGICOS

Em linguagem de programação, este tipo de dado é também conhecido como booleano, devido à contribuição de Boole à área da lógica matemática.

Este tipo de dado é usado para representar dois únicos valores: - Verdadeiro ou Falso.

Em algoritmos, os valores lógicos sempre são apresentados estando entre os caracteres de ponto (.), conforme exemplos a seguir:

.V. (valor lógico verdadeiro)
.F. (valor lógico falso)

Cada tipo de dado que apresentamos acima, necessitam de uma certa quantidade de memória para armazenar as informações e estes dados serão armazenados no que chamamos de variáveis de memória. Uma variável, possui basicamente três atributos:

- Nome
- Tipo de dado que irá armazenar
- Informação a ser armazenada

Todas as variáveis possuem um nome que tem a função de identifica-la e diferencia-la das demais utilizadas no mesmo sistema. Este nome é formado através de um conjunto de regras, que poderá ser diferente, dependendo da linguagem que o programador estiver utilizando, sendo que a maioria das linguagens adota as regras seguintes:

- O nome de uma variável deve necessariamente começar com uma letra;
- Um nome de variável não teve conter símbolos especiais (exceto o sublinha);
- O nome de uma variável deve, por questões de facilitar a interpretação do sistema, lembrar a informação que irá armazenar.

Veja a seguir exemplos de variáveis:

- Nome_funcionario
- Nome_cliente
- Salario
- Cep

Observe que é fácil identificar o objetivo e a informação que serão armazenadas nas variáveis acima. Observe outros exemplos de variáveis:

- Xyu
- Iausio

Você saberia identificar o tipo de informação a ser armazenada e o objetivo da mesma nas duas variáveis acima citadas?

Desta forma fica fácil entender o porque das regras citadas.

É regra a definição das variáveis a serem utilizadas durante o algoritmo antes de sua utilização, desta forma o compilador irá reservar um espaço na memória para o trabalho com esta.

Esta definição em algoritmos se dará através da seguinte sintaxe:

VAR <nome_da_variável> : <tipo_da_variável>

No exemplo acima, estamos definindo apenas uma variável. Caso você queira definir um conjunto de variáveis deverá utilizar a seguinte sintaxe:

VAR <lista_de_variáveis> : <tipo_das_variáveis>

Nesta definição, deveremos seguir as regras seguintes:

- VAR é a palavra chave, que deverá ser utilizada uma única vez na definição das variáveis e antes do uso das mesmas;
- Variáveis de tipos diferentes, deverão ser declaradas em linhas diferentes;
- Em uma mesma linha, quando quisermos definir variáveis de mesmo tipo, deveremos usar o símbolo de ponto e vírgula (;) para separar as mesmas.

Observe o exemplo a seguir:

VAR NOME:LITERAL[30]

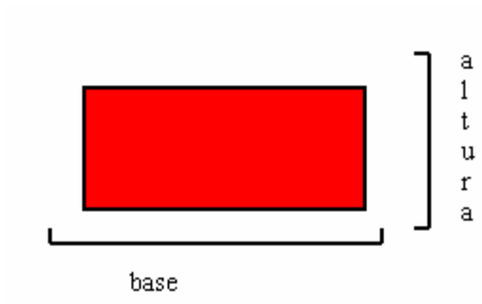
SALARIO:REAL ; HORAS_EXTRAS:REAL

SALARIO_FAMILIA : LÓGICO

No exemplo acima, declaramos a variável “NOME” do tipo “String” com capacidade máxima para 30 caracteres. As variáveis “salario” e “horas_extras” foram definidas como “real”, ou seja, aceitam números com parte decimal. A variável “salario_familia” é do tipo lógica, ou seja, apenas valores do tipo verdadeiro (.T.) ou falso (.F.).

Na lógica de programação uma expressão tem o mesmo objetivo/conceito do termo expressão da matemática comum, ou seja, um conjunto de variáveis e constantes que se relacionam por meio de operadores aritméticos. Este conjunto de expressão e operadores aritméticos (soma por exemplo), formam uma fórmula que, após solucionada fornecem um resultado em específico.

Observe o gráfico a seguir:



Neste caso, a fórmula para se calcular a área de um retângulo é igual ao produto de sua altura por sua base, poderemos então montar a fórmula como se segue:

AREA = altura * base

Observe que no exemplo acima, utilizamos três variáveis: altura, base e área, sendo que o resultado final é armazenado na variável AREA, ou seja, local de memória onde será armazenado o resultado da avaliação da expressão e o operador é o sinal de ‘*’ que em linguagem de computador representa a multiplicação.

Já que falamos em operadores, vamos ser um pouco mais claros neste tópico. Os operadores são elementos ativos que tem influência sobre os operandos e através de sua ação resultam um determinado valor. No exemplo acima, os operandos seriam as variáveis “altura” e “base” e o operador de multiplicação o “*”.

Em lógica de programação, os operadores podem ser classificados em dois grupos:

- Binários : quando trabalham sobre os operandos em uma expressão. Os mais conhecidos são os símbolos + - * /, que representam a soma, subtração, multiplicação e divisão respectivamente.
- Unários : quando tem influência sobre um único operando, indicando por exemplo que este se trata de um número negativo e não vem acompanhado um um outro operando, exemplo: ao representar o número quatro negativo, podemos utilizar (-4). Isso não quer dizer que queremos subtrair o quatro de um outro valor.

Assim como classificamos os operadores, podemos também classificar as expressões quanto ao seu tipo, conforme mostra a lista a seguir:

- Expressão aritmética - quando o resultado de sua avaliação for um número, podendo este ser positivo ou negativo assim como inteiro ou real;
- Expressão lógica - quando o resultado de sua avaliação for um valor lógico, ou seja, verdadeiro (.T.) ou falso (.F.);
- Expressão literal - quando o resultado de sua avaliação for um valor literal.

Regras Gerais sobre Expressões:

- Em expressões aritméticas, existe prioridade entre operadores em uma mesma operação, e esta prioridade define a ordem pela qual os operadores serão executados. Nesta ordem, citamos:

Primeiro é observado o estado unário de um determinado valor/variável;

Em seguida, dentro de uma operação, num grau de prioridade de 3 à 1, estão respectivamente os operandos + - * / **

- Em expressões lógicas, em um grau de prioridade de 3 à 1 temos os operadores .or. .and. .not..
- Em expressões lógicas, o operador .NOT. sempre irá inverter o valor de seu operando. Exemplo: **.NOT. .T. = *.F.***
- Em operações lógicas com o operador .OR. basta que um dos valores comparados seja verdadeira.
- Em operações lógicas com o operador .AND. os dois valores comparados tem que ser verdadeiros.

Após dominarmos os assuntos anteriormente tratados, passaremos a estudar as instruções primitivas, ou seja, comandos básicos que executam tarefas consideradas essenciais para a execução de qualquer programa de computador. Um exemplo deste tipo de instrução são aquelas responsáveis pela comunicação do operador com o computador por meio do teclado (entrada de dados) ou ainda a impressão de um relatório (saída de dados “sem contar com a parte estética do relatório, alinhamento de colunas, etc...).

Toda linguagem de programação tem por obrigação possuir instruções primitivas, pois sem estas, não existiria comunicação com periféricos.

Antes de mais nada, você saberia diferenciar periféricos de entrada e de saída de dados? A resposta é simples, periféricos de entrada são aqueles responsáveis pela passagem de dados do mundo externo para a memória do computador, como por exemplo o teclado, unidade de CD-ROM, etc... Já os periféricos de saída recebem os dados do computador para um outro equipamento externo, como por exemplo o monitor de vídeo, impressora, winchester, etc...

Toda instrução, primitiva ou não possui uma sintaxe, ou seja, uma regra que deverá ser seguida para a construção de seu código, e caso esta não seja obedecida, o seu programa pode simplesmente não funcionar.

Devemos saber também que o conjunto de ações que serão realizadas pelo computador após a execução de um determinado comando é conhecida como semântica.

Vamos agora descrever algumas instruções primitivas que serão de grande utilidade nos próximos módulos:

Instrução Primitiva de atribuição

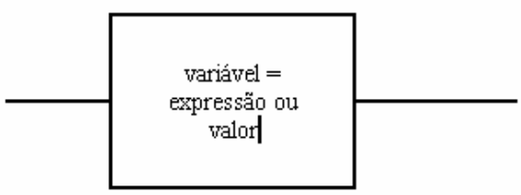
É a forma pela qual armazenamos um determinado valor em uma variável. Sua sintaxe básica é:

Nome_da_variável = <valor> ou <expressão>

A semântica de uma atribuição consiste em:

- Avaliação da expressão;
- Armazenamento do valor resultante em uma variável (posição de memória).

Em um fluxograma, uma instrução de atribuição pode ser representado da forma seguinte:



Instrução primitiva de saída de dados

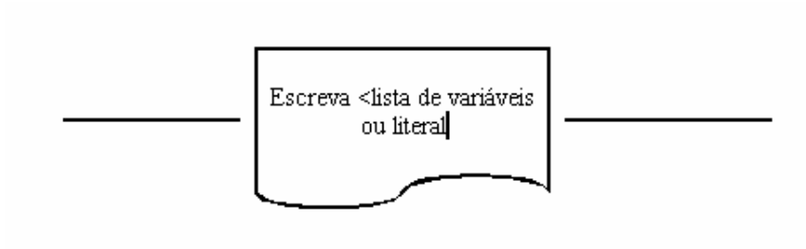
Do que serviria um algoritmo que tenha o resultado de um determinado calculo e este não for enviado ao operador a nível de resultado? Para isso temos a instrução primitiva de saída de dados.

A sintaxe mais comum neste sentido é apresentado a seguir:

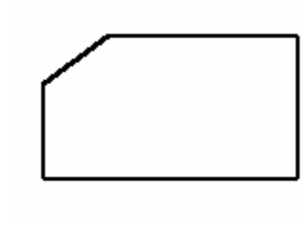
Escreva <lista_de_variáveis> ou <literal>

Desta forma, Escreva passa a ser uma palavra reservada e não poderemos utilizar a mesma como nome de variável. Quando você for “exibir” uma lista de variáveis, as mesmas deverão vir separadas por vírgula.

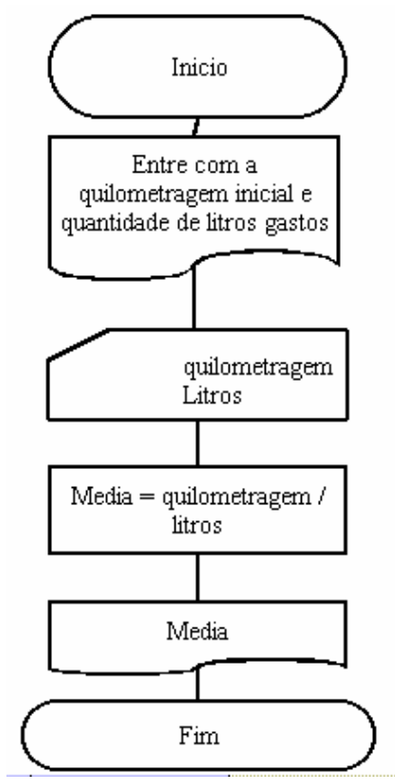
Em uma fluxograma, uma instrução de saida de dados, pode ser representada da forma que segue:



A instrução primitiva de entrada de dados no fluxograma é representado pelo símbolo:



Vamos então criar o exemplo de duas formas, fluxograma/pseudocódigo:



Algoritmo media

Var

quilometragem,litros,media : real

Inicio

Escreva “Entre com a quilometragem inicial e a

Quantidade de litros gastos”

Leia quilometragem

Leia litros

Media = quilometragem/litros

Escreva media

Fim.

Observe que no pseudocódigo, quando queremos enviar uma expressão como saída, a mesma é colocada dentro de aspas, porém quando queremos enviar o conteúdo de uma variável, esta deverá estar fora de aspas.

Exercícios:

001) Desenhe o fluxograma do algoritmo a seguir, que tem a função de calcular e exibir o valor total da compra de um determinado produto, que tem o resultado baseado no valor unitário do produto multiplicado pela quantidade adquirida:

```
Algoritmo precototal
Var
        quantidade,vl_unitario,vl_total
Inicio
        Escreva "Entre com a quantidade e o valor unitário do produto"
        Leia quantidade
        Leia vl_unitario
        Vl_total = vl_unitario * quantidade
        Escreva "Valor total da compra"
        Escreva vl_total
Fim.
```

002) Desenhe o fluxograma do algoritmo a seguir, que tem a função de calcular o valor liquido de compra de um determinado produto, sabendo-se que sobre o valor bruto será dado um desconto "x" a ser obtido através de uma instrução primitiva de entrada de dados.

```
Algoritmo desconto
Var
        desconto,vl_bruto,vl_liquido
Inicio
        Escreva "Entre com o valor do produto e o índice de desconto"
        Leia vl_bruto
        Leia desconto
        vl_liquido = (vl_bruto-((vl_bruto*desconto)/100))
        Escreva "Valor liquido do produto"
        Escreva vl_liquido
Fim.
```

Até o momento, utilizamos apenas funções de entrada e saída de dados, sem exercer nenhum controle sobre as mesmas.

Porém, em qualquer programa de computador, independente da linguagem de programação a ser utilizada, dificilmente deixaremos de usar rotinas de controle de fluxo e rotinas de testes, também conhecidas como estruturas de decisão.

Imagine a seguinte situação : em uma empresa, será solicitado o salário de um determinado funcionário, para se calcular seu novo salário, sendo que, se este tiver um salário superior a R$ 300,00, o reajuste será de 5%, caso contrário o reajuste será de 8%.
Observe que teremos que testar o valor do salário, para saber o índice a ser aplicado.

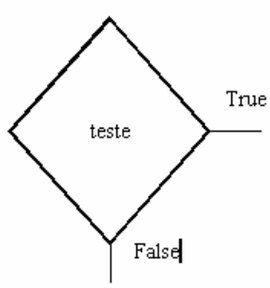
Vamos criar então um pseudocódigo com este objetivo:

```
Algoritmo testa_salario
Var
        Salario : real
        Salario_final : real
Inicio
        Leia salario
        Se salario <= 300
                então
                        Salario_final=(salario*1,08)
                senão
                        Salario_final=(salario*1,05)
        Fim_se
        Escreva "Salario_final"
Fim.
```

Observe que, durante a execução do pseudocódigo, após obtermos, através de uma instrução de entrada de dados, o valor do salário do funcionário, efetuamos um teste "Se", que possui duas condições, uma verdadeira e uma falsa. As instruções que serão executadas no caso de um teste verdadeiro, devem estar abaixo da cláusula "Então", já as instruções que serão executadas no caso de um teste falso, devem estar abaixo da cláusula "senão". A instrução "Fim_se", termina o bloco de testes. Observe a sintaxe de uma estrutura de decisão a seguir:

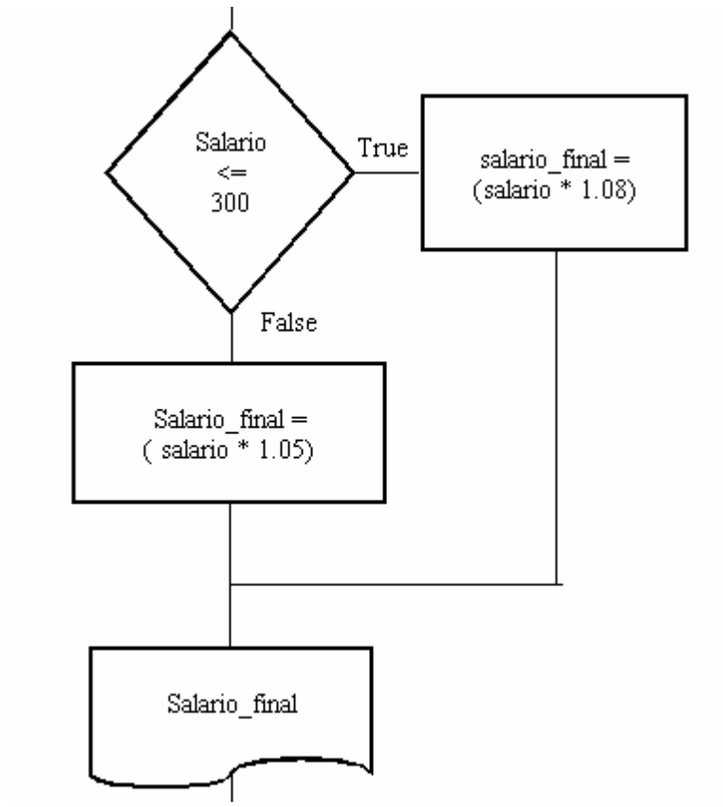
Se <teste>
Então
Lista de comandos a serem executados caso teste=verdadeiro
Senão
Lista de comandos a serem executados caso teste=falso.
Fim_se

Em fluxograma, uma estrutura de decisão é representada pelo símbolo:



Os símbolos que estiverem na direção de “True” serão executados caso o teste seja verdadeiro. Já os símbolos que estiverem na direção de “false” serão executados caso o teste tenha um resultado falso.

Observe o trecho do pseudocódigo acima, representado no fluxograma:

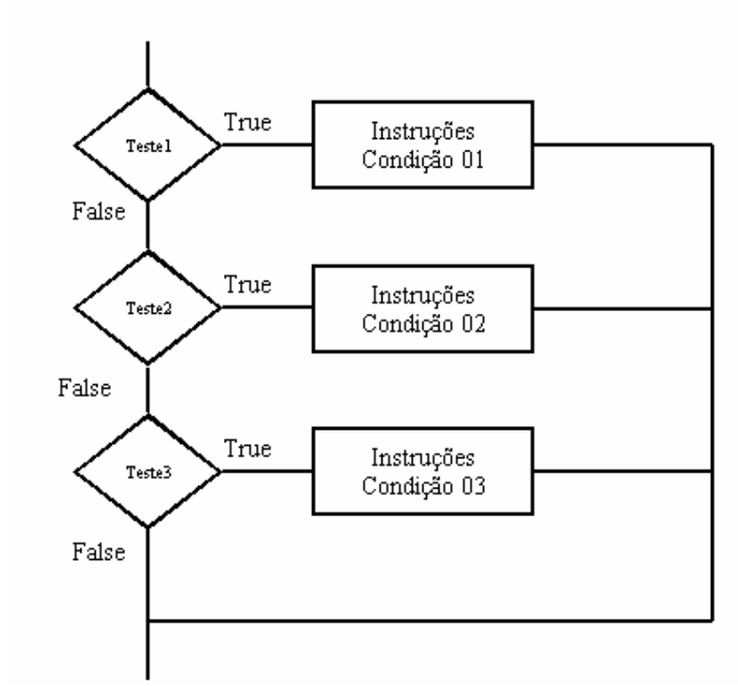


Existem casos em que não basta ter apenas as “direções” verdadeiro e falso para tomar uma determinada decisão, e sim, uma série de testes sobre um mesmo bloco. Este tipo de estrutura é chamada de estrutura de decisão do tipo “ESCOLHA”.

Observe a seguir a sintaxe da estrutura de decisão do tipo escolha em pseudocódigo:

```
Escolha
        Caso <condição 1>
                Lista de instruções caso condição 1 = verdadeira
        Caso <condição 2>
                Lista de instruções caso condição 2 = verdadeira
        Caso <condição 3>
                Lista de instruções caso condição 3 = verdadeira
        Senão
                Lista de instruções caso todas condições = falsa
Fim_escolha.
```

No fluxograma, o símbolo que representa cada uma das condições acima, é o mesmo que o símbolo que representa a estrutura de decisão. Veja a sintaxe da estrutura de decisão de escolha no fluxograma:



Vamos imaginar um exemplo parecido, de reajuste de salário, porém teremos as seguintes situações para reajuste:

Salários inferiores a R$ 400,00 = reajuste de 9%
Salários iguais a R$ 400,00 e inferiores a R$ 800,00 = reajuste de 7%
Salários iguais a R$ 800,00 e inferiores a R$ 1000,00 = reajuste de 5%
Fora das faixas supra citadas = reajuste de 3%

Veja o pseudocódigo do exemplo citado:

```
Algoritmo testa_salario2
Var
        Salario : real
        Salario_final : real
Inicio
        Leia salario
        Escolha
                Caso salario < 400
                        Salario_final=(salario*1.09)
                Caso salario >= 400 .e. salario<800
                        Salario_final=(salario*1.07)
                Caso salario>=800 .e. salario<1000
                        Salario_final=(salario*1.05)
                Senão
                        Salario_final=(salario*1.03)
        Escreva “Salario_final”
Fim.
```

Observe que, temos aqui uma novidade, ou seja, em uma única condição, estamos na verdade realizando dois testes, isto porque usamos o operador lógico “AND” “.e.”, ou seja, se salário menor ou igual a 400 e salário menor que 800 faça isso... assim sucessivamente.

Quando usamos o operador lógico “.e.”, os testes, neste caso dois, que formam o caso, devem ser verdadeiros. O outro operador lógico “.ou.”, retorna verdadeiro caso um dos testes que formam o conjunto satisfaça a uma determinada condição.

Como exercício, crie o fluxograma do pseudocódigo visto acima.

Exercícios:

1.) Faça um alg/flux que leia a altura de duas pessoas (A e B) e verifique qual é a maior e exiba com uma mensagem qualquer qual é o maior. Ex: “A é maior que B”

2.) Faça um alg/flux que leia os lados de um triângulo retângulo e exiba sua hipotenusa. Se a hipotenusa for maior que 100 escreva, hipotenusa muito grande, caso contrário exiba hipotenusa pequena.

3.) Faça um alg/flux que permita a entrada da temperatura em Celsius, calcule e exiba as temperaturas em kelvin e em farenheit. Se a temperatura em Celsius for maior ou igual a 30 graus exiba “Tempo bom para praia”; se estiver entre 20 a 30 escreva “Tempo Instável” e se for menor que 20 escreva “Tempo bom para dormir”.

4.) Faça um alg/flux que permita a entrada de um número qualquer e exiba se este número é par ou ímpar. Se for par exiba também a raiz quadrada do mesmo; se for ímpar exiba o número elevado ao quadrado.

5.) Faça um alg/flux para calcular uma equação do 2º Grau. Se delta for negativo, escreva “Não existem raízes reais”; se delta for igual a zero, faça o cálculo e exiba apenas uma raiz e se delta for maior que zero, calcule e exiba as duas raízes.

Estruturas de Repetição

Em linguagem de programação, as estruturas de repetição são conhecidas como laços (loops).

Trabalhamos basicamente com dois tipos de estruturas de repetição:

Loops contados : quando se tem, de forma prévia, o número de vezes que uma determinada sequência de comandos será executada.

Loops condicionais : aqueles que não possuem um número pré-determinado de vezes que a estrutura de comandos será executada, porém este número estará vinculado a uma determinada condição.

Em pseudocódigo, utilizamos a seguinte sintaxe para loops contados:

Para <variavel> de <valor inicial> até <valor final> incr de <numero> faça
<sequencia de comandos>
Fim_para

Onde:

A variável é aquela que irá receber a contagem. Os valores iniciais e finais representam onde começa e termina a contagem da repetição. O número de incre (incremento) estipula a forma da contagem, como por exemplo, incrementos de 1 em 1, 2 em 2 .... e após o faça, a sequência de comandos que será executada durante o loop.

No fluxograma, utilizamos a seguinte sintaxe para representar loops contados:

A lógica de trabalho do laço contado é apresentada a seguir:

No inicio da leitura do código/fluxo, o valor inicial é atribuído à variável;

O valor da variável é comparado ao valor final;

Se o valor da variável for maior que o valor final, a sequência de comandos que faz parte do laço não é executado e o controle do processo passa para a próxima linha após o final do laço (FIM_PARA) ou para o símbolo ao lado (no caso de fluxograma);

Se o valor da variável for menor ou igual ao do valor final, a sequência de comandos é executada e, ao final do último comando, o valor do incremento é adicionado a variável e retorna-se à comparação entre variável e valor_final, repetindo todo o processo anterior.

Vale lembrar que o incremento não é de declaração obrigatória, e, caso não seja declarado, assume automaticamente o valor 1.

Já os loops condicionais são aqueles cujo conjunto de comandos em seu interior é executado até que uma determinada condição seja satisfeita.

O pseudocódigo mais comum utilizado neste sentido é o apresentado a seguir:

Enquanto <condição>
    <sequência de comandos>
Fim_enquanto

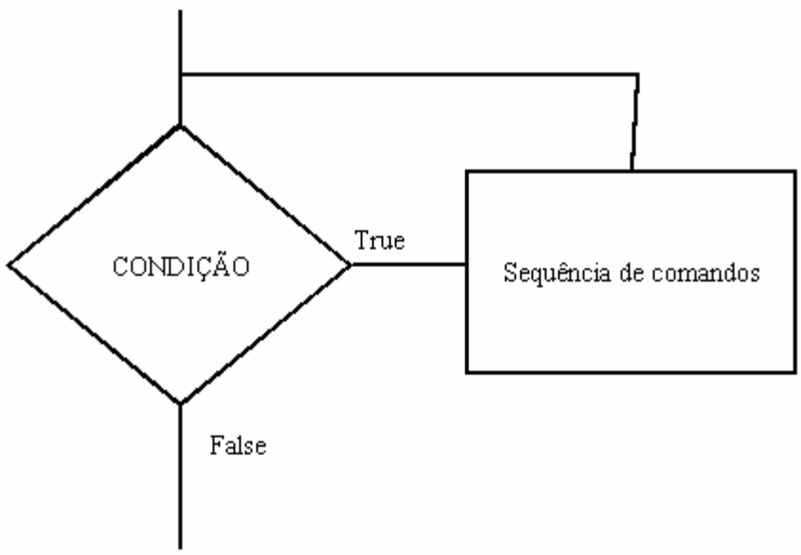
A sintaxe de execução deste é:

No inicio da execução do enquanto, a condição é testada;

Se o resultado do teste for verdadeiro, a sequência de comandos é executada e ao término desta, retorna-se ao teste da condição;

Sendo a condição falsa, o controle da aplicação passa para a próxima linha após o Fim_enquanto.

No fluxograma, poderemos representar este tipo de operação da forma que segue:



Em resumo, neste tipo de operação, a execução somente abandonará o laço quando a condição for falsa.

Veja um exemplo a seguir, utilizando-se de pseudocódigo, onde iremos apresentar a soma de uma contagem de números de 1 à 5:

Algoritmo soma

Var
    Soma : real
    Contador : inteiro

Inicio
    Soma = 0
    Escreva "A soma dos números de 1 à 10 é igual a"
    Para contador **de** 1 **até** 10 **faça**
        Soma = soma + contador
    Fim_para
    Escreva soma
Fim.

No exercício acima, teremos como resultado a expressão :

A soma dos números de 1 à 10 é igual a 55

Desenhe o fluxograma do pseudocódigo acima descrito.

O conjunto de comandos é um conjunto de comandos simples, podem ser inclusive de instruções primitivas.

Por exemplo, temos uma determinada situação:

Vamos testar a região na qual um determinado cliente tem o seu domicílio e, dentro do estado na qual este cliente reside naquela região, teremos uma tarifa de postagem diferenciada.

Na verdade, teremos testes dentro de testes e cada um com um determinado procedimento.

Exercícios propostos

- **Exercício 001 :**

Escreva um pseudocódigo que irá receber a entrada de dois números e irá retornar como saída a entrada (número) de maior valor.

- **Exercício 002 :**

Escreva um pseudocódigo na qual o usuário irá ter a oportunidade de entrar com 5 números e o sistema irá gerar como resultado a soma dos 5 números entrados pelo usuário.

- **Exercício 003 :**

Uma empresa irá oferecer um reajuste salarial, cujo percentual será decidido através das seguintes situações:

- Funcionários do sexo masculino com tempo de casa superior ou igual a 10 anos terão direito a um reajuste de 5%;
- Funcionários do sexo masculino com tempo de casa inferior a 10 anos terão direito a um reajuste de 3%;
- Funcionários do sexo feminino com tempo de casa superior ou igual a 8 anos terão direito a um reajuste de 5%;
- Funcionários do sexo feminino com tempo de casa inferior a 8 anos terão direito a um reajuste de 3%;
- Funcionários em qualquer uma das situações acima, porém com mais de um dependente terá ainda um reajuste de 2% sobre o salário reajustado anteriormente.

- **Exercício 004 :**

Escreva um fluxograma na qual o usuário dará a entrada de 10 números e ao final o sistema irá exibir o maior número entrado.

Variáveis Indexadas

O tema deste módulo será o das variáveis indexadas. Com uso frequente na programação convencional, variável indexada é um conjunto de variáveis do mesmo tipo, referenciadas pelo mesmo nome e que armazenam valores distintos.

A sua distinção se dá pela divisão da variável, possuindo esta uma posição de armazenamento dentro de seu conjunto, e esta é realizada por meio de índices, daí o nome indexada. Existem dois tipos de variáveis indexadas:

- Vetor : quando a variável indexada possui um único índice;
- Matriz : quando a variável indexada

O número de índices dentro de uma variável é denominado dimensão.

Para se definir uma variável indexada, utilizamos a seguinte sintaxe:

Var

Nome_variável : **conjunto[dimensão1,dimensão2...]** tipo_variável

...

...

Veja a seguir alguns exemplos de definição de variáveis indexadas:

Nome : conjunto[5] String[50]

No exemplo acima, criamos na verdade cinco variáveis nomes, porém elas estão divididas da seguinte forma:

Nome[1] , Nome[2], Nome[3], Nome[4], Nome[5]

Sendo que cada uma das dimensões, poderá armazenar um valor diferenciado, e cada uma delas, poderá ter até 50 caracteres (literal[50]).

Salario : conjunto[5] Real

A variável Salario é do tipo Real, e possui 5 dimensões, assim como a variável Nome.

Os exemplos acima são do tipo Vetor, abaixo veremos uma variável indexada do tipo matriz:

CodigoProduto : conjunto[3,10] Inteiro

No exemplo anterior, teremos a variável CodigoProduto, dividida nas seguintes dimensões:

CodigoProduto[1,1]CodigoProduto[2,1]CodigoProduto[3,1]
CodigoProduto[1,2]CodigoProduto[2,2]CodigoProduto[3,2]
CodigoProduto[1,3]CodigoProduto[2,3]CodigoProduto[3,3]
CodigoProduto[1,4]CodigoProduto[2,4]CodigoProduto[3,4]
CodigoProduto[1,5]CodigoProduto[2,5]CodigoProduto[3,5]
CodigoProduto[1,6]CodigoProduto[2,6]CodigoProduto[3,6]
CodigoProduto[1,7]CodigoProduto[2,7]CodigoProduto[3,7]
CodigoProduto[1,8]CodigoProduto[2,8]CodigoProduto[3,8]
CodigoProduto[1,9]CodigoProduto[2,9]CodigoProduto[3,9]
CodigoProduto[1,10]CodigoProduto[2,10]CodigoProduto[3,10]

Vamos comparar as variáveis indexadas a um Bloco de uma quadra residencial. O bloco é único, porém está dividido em diversos apartamentos (dimensões) e em cada apartamento reside um morador diferente (valor armazenado).

Quando estamos trabalhando com variáveis indexadas, temos que obrigatoriamente especificar o índice da variável na qual queremos trabalhar. Nos exemplos acima, não poderíamos nos referir apenas a variável NOME, mas sim a NOME[x], onde o "x" seria uma dimensão válida, neste caso de 1 à 5.

Vamos exibir um exemplo onde iremos preencher os valores para a variável nome em suas diversas dimensões e em seguida listar os mesmos:

```
Algoritmo Nomes
Var
        Nome : conjunto[5] String(50)
        Conta : inteiro
        Lista : inteiro
Inicio
        Para conta de 1 até 5 faça
                Leia Nome[conta]
        Fim_para

        Para lista de 1 até 5 faça
                Escreva nome[lista]
        Fim_para
Fim.
```

Exercício 001 : Escreva um pseudocódigo que irá Ler e armazenar em variável indexada 10 números e após o final da leitura, irá exibir a soma dos 10 números armazenados nas dez dimensões da variável indexada.

Exercício 002 : Escreva um pseudocódigo que irá ler e armazenar em variável indexada 50 números e após o final de leitura, irá apresentar o número da dimensão e o valor armazenado da dimensão de variável indexada que conterá o maior valor lido.

Quando trabalhamos com variáveis indexadas, poderemos entre outros criar "índices", ou seja, ordenar os vetores para que eles sejam apresentados em uma determinada ordem. Este procedimento, também é conhecido, em sua forma mais simples como "Bolha de Classificação". Sua lógica consiste na leitura de todos os elementos de um vetor, comparando-se os valores de seus elementos vizinhos, e neste momento é empregada a seguinte sequência:
Será realizada uma varredura a menos do total de elementos do vetor;

- Na primeira varredura, verificamos que o elemento do vetor já se encontra em seu devido lugar;

- Na segunda varredura, o procedimento é análogo à primeira varredura e vai até o último elemento;
- Ao final do procedimento, o vetor estará classificado segundo o critério escolhido.

Vamos à prática, no exemplo a seguir, iremos realizar a leitura de 5 números e em seguida classificá-los em ordem crescente:

```
Algoritmo classificação
Var
        numero : conjunto[50] de inteiro
        A,B : inteiro
        Aux : Real
Inicio
        Para A de 1 até 50 faça
                Leia numero[A]
        Fim_para
        B=50
        Enquanto B > 1 Faça
                Para A de 1 até ( B-1 ) faça
                        Se numero[A] > numero[A+1}
                                Então
                                        Aux=numero[A]
                                        numero[A]=numero[A+1]
                                        numero[A+1]=Aux
                        Fim_se
                Fim_para
                B=B-1
        Fim_enquanto
        Escreva "Abaixo, a listagem ordenada dos números entrados"
        Para A de 1 até  50 faça
                Escreva numero[A]
        Fim_para
Fim.
```

Um outro processo que poderá ser utilizado em programação, é a pesquisa sequencial, ou seja, serão verificados todos os componentes de um vetor, para verificar se nestes estão armazenados um determinado valor. Exemplo:

Iremos efetuar a leitura de 100 nomes, em seguida, será solicitado um nome qualquer e, iremos verificar se, dentre os 100 nomes entrados, existe o nome na qual foi solicitada a pesquisa. Veja este algoritmo a seguir:

```
Algoritmo pesquisa
Var
        nome : conjunto[100] de literal[50]
        pesquisado : literal[50]
        contador : inteiro
        encontrado : lógico
Inicio
        Para contador de 1 até 100 faça
                Leia nome[contador]
        Fim_para
        Leia pesquisado
        Escreva "Aguarde ... pesquisando no banco de dados"
        contador=1
        encontrado=.F.
        Enquanto contador<101 .E. .NÃO. encontrado Faça
                Se nome[contador]=pesquisado
                        Então
                                Encontrado=.T.
                        Senão
                                Contador=contador+1
                Fim_se
        Fim_enquanto
        Se encontrado
                Então
                        Escreva "O valor está contido no banco de dados"
                Senão
                        Escreva "O valor não existe no banco de dados"
        Fim_se
Fim.
```

SubAlgoritmos

Em determinadas situações, devido à complexidade de alguns algoritmos, é necessária a divisão do mesmo em diversas partes, para que assim, o sistema tenha uma operação precisa. Essa divisão de tarefas é denominada de "Subalgoritmos". Os subalgoritmos nada mais são do que rotinas que possuem uma função específica. Vamos imaginar a seguinte situação:

Em um sistema comercial, em várias etapas temos que verificar se o número do CPF do cliente foi digitado de forma correta. Imaginando que tal situação se repete por 38 vezes no sistema. Teríamos que escrever 38 vezes o mesmo trecho de código? Não, para isso teríamos uma rotina "subalgoritmo" que teria esta função. Este subalgoritmo teria um nome e, sempre que fosse necessária a verificação do CPF, bastaria invocar "chamar" este subalgoritmo.

De um modo geral, os subalgoritmos são importantes devido aos seguintes aspectos:

- Estruturação de algoritmos, facilitando assim a detecção de erros. Imagine se a rotina de verificação do CPF do cliente tiver um erro lógico. Em uma primeira hipótese, teríamos que corrigir a mesma em 38 pontos diferentes do sistema, porém utilizando subalgoritmo, teríamos apenas que realizar uma correção em um ponto do sistema;
- Modularização de sistemas, que justamente é utilizada para a reutilização de rotinas "subalgoritmos" em vários pontos do algoritmo principal;
- Subdivisão de algoritmos extensos, facilitando assim a sua compreensão.

O esquema de um algoritmo e seus subalgoritmos pode ser observado a seguir:

Algoritmo (nome do algoritmo)
Var
    Definição das variáveis globais
<definição dos subalgoritmos>
Inicio
    <estrutura do algoritmo principal>
Fim.

Quando o sistema está executando o algoritmo principal e, é invocado algum subalgoritmo (através de um nome dado ao mesmo), a execução deste é interrompida e, o sistema passa a executar os comandos constantes do subalgoritmo. Ao final desta execução, o sistema retorna automaticamente para o algoritmo principal a partir do ponto onde foi realizada a chamada do subalgoritmo.

Um subalgoritmo é composto por:

- **Cabeçalho :** onde é definido o nome do subalgoritmo e as variáveis que serão utilizadas pelo mesmo;
- **Corpo :** onde são especificadas as instruções do subalgoritmo.

Os subalgoritmos podem ser de dois tipos:

- Funções;
- Procedimentos.

O subalgoritmo tipo função é aquele na qual um valor é calculado com base em outros valores, normalmente passados pelo algoritmo principal.

O subalgoritmo tipo procedimento é aquele que retorna zero ou mais valores ao algoritmo que o invocou, sendo que, ao contrário das funções, esses valores não são explícitos, ou seja, a

chamada de procedimentos nunca é realizada ao meio de expressões é só é realizada em comandos isolados dentro do algoritmo, como instruções de entrada e saída de dados.

Veja a seguir um algoritmo e seu subalgoritmo para se calcular a raiz quadrada de um determinado número:

Algoritmo Raiz_quadrada
Var num1, num2 : **real**
Função Raiz(numero:**real) : real**
Var calc : **real**
Inicio
    Calc = SQR(numero)
    Retorne Calc
Fim
Inicio
    Escreva **"**Numero para Raiz Quadrada"
    Leia num1
    Num2 = Raiz(num1)
    Escreva **"**Raiz quadrada é igual a:", num2
Fim

Com certeza, teria sido mais fácil executar o cálculo de raiz quadrada diretamente no algoritmo principal, porém o fizemos assim para que você tenha uma idéia de como funciona um subalgoritmo, ou seja, a passagem de valores de uma rotina principal, para uma rotina secundária e o retorno do valor resultante deste processamento.